fastmcp_gateway_project...
Zip Archive

I feel like my project has started to become too big that I'm having a hard time understanding portions of it. I'm focusing on the Oberservability part right now and I want to make sure the implementation stays the same, but I really need a much simpler version of the project where I can follow the code easier. Please generate. Remember, keep all the important opentelemetry logic and ways needed to have the code abstracted enough, using pydantic, and not having too much bloat

I'll turn the uploaded project into a smaller "observability-first" version: keep the OpenTelemetry span/tool wrapping behavior, Pydantic request/response/config models, and a clean abstraction boundary, while removing extra prototype bloat.

I found the key observability path: FastAPI request span → session span → engine span → agent invoke span → child `tool.invoke ...` spans. The important part to keep is the MCP tool wrapping at load time, because that is what records each real tool call without editing every server.

Thought for 3m 49s

Generated a slimmed-down observability-focused version of your project and packaged it here:

Download the simplified project ZIP

I kept the important pieces:

- OpenTelemetry setup and FastAPI instrumentation
- Request/session/engine/tool span structure
- MCP tool wrapping at load time
- `ContextVar`-based request-level tool call collection
- Pydantic settings, API schemas, metrics, and telemetry models

- JSONL metrics output
- Minimal FastMCP gateway with calculator and quote tools

I removed the bigger pieces that were making the project harder to follow:

- frontend
- benchmarking scripts
- quality-scoring heuristics
- data analyst server
- sample data
- filesystem skills
- extra gateway/prototype bloat

I also syntax-checked the generated Python files successfully.

can you give me a simple step by step how this new code works (focusing on the observability piece)

I'll walk through it from app startup to a single chat request, and I'll keep the focus on where traces, spans, and per-request tool records are created.

Thought for 22s >

Here's the simple observability flow for the slim project.

1. App starts in `backend/main.py`

When you run:

Bash

FastAPI loads this:

Python Run

Inside `create_app()`:

```
setup_otel(...)
instrument_fastapi(...)
```

This is the first observability setup point.

What happens here

`backend/observability/otel.py` configures OpenTelemetry.

If this is in `.env`:

```env
```

then tracing is basically off.

If this is enabled:

```env
```

then OpenTelemetry creates traces/spans.

If you do not provide an OTLP endpoint, spans print as long JSON in the terminal:

```
OTEL_ENABLED=true
OTEL_EXPORTER_OTLP_TRACES_ENDPOINT=
```

That is the long JSON string you saw before.

If you provide an OTLP collector endpoint, spans are exported there instead:

```env
```

2. FastAPI gets automatic request spans

This line matters:

Python Run

That tells OpenTelemetry:

> "Every HTTP request should get a parent span."

So when someone calls:

http

OpenTelemetry creates a span for that request.

Then your own code adds attributes to that existing request span:

```
request_span = trace.get_current_span()
request_span.set_attribute("chat.session_id", req.session_id)
request_span.set_attribute("chat.user_message_len", len(req.message))
```

So the top-level request span knows things like:

```
chat.session_id
chat.user_message_len
chat.success
chat.total_response_time_ms
chat.tools_used
chat.tool_call_count
```

This happens in:

```
backend/main.py
```

3. The app starts the MCP runtime

During app startup, this happens:

```
runtime = MCPRuntime(settings=settings)
await runtime.start()
```

This lives in:

```
backend/mcp_runtime.py
```

The MCP runtime knows about the gateways:

```
self.connections = {
    "core": {"transport": "http", "url": self.settings.core_gateway_url},
}
```

So the backend talks to the core FastMCP gateway, not directly to each individual tool server.

The gateway is here:

```
gateways/core_gateway.py
```

It mounts the calculator and quote servers:

```
gateway.mount(..., namespace="math")
gateway.mount(..., namespace="quote")
```

So the flow is:

```
Backend
  -> Core Gateway
      -> Calculator MCP Server
      -> Quote Counter MCP Server
```

4. Tools are loaded once and wrapped once

This is one of the most important parts.

In `backend/agent.py`, startup calls:

Python Run

That goes into:

`backend/mcp_runtime.py`

Inside `get_all_tools()`:

Python Run

So tool loading itself gets a span:

`mcp.get_all_tools`

Then for each gateway alias, this runs:

Python Run

Inside `_ensure_loaded()`:

Python Run

So each gateway load gets another span:

`mcp.ensure_alias_loaded`

Then tools are loaded:

Python Run

Python Run

Then each tool is wrapped:

```python
wrapped_tools = [wrap_tool_for_tracing(tool, alias=alias) for tool in raw_tools]
```

Python Run

That wrapper lives here:

```
backend/observability/tool_wrapper.py
```

Important point

The tool wrapper does **not** create extra tool calls.

It does this:

```
Original tool function
        ↓
Wrapped tool function
        ↓
Real original tool still runs
```

It simply adds tracing around the real call.

So this:

```
call add tool
```

becomes:

```
start span
call real add tool
record duration/success/error
end span
```

It does not call the tool twice.

5. The Deep Agent receives the wrapped tools

Still in `backend/agent.py`, after tools are loaded:

```
self.agent = create_deep_agent(
    model=ChatOpenAI(...),
    tools=tools,
    system_prompt=...
)
```

The important thing is that `tools` are already wrapped.

So the agent does not know or care about observability.

The agent just sees normal tools.

But when it calls one, the wrapper records the span.

That is a clean abstraction.

6. A user sends a chat request

User calls:

```http
```

FastAPI enters this function:

```
@app.post("/chat")
async def chat(req: ChatRequest) -> ChatResponse:
```

This function starts measuring total request time:

```Python
```

Run

Then it passes the request to the session layer:

Python Run

7. Session layer creates a `session.chat` span

This happens in:

```
backend/session_store.py
```

The session layer starts a span:

Python Run

This creates:

```
session.chat
```

This span records:

```
session.id
session.is_new
message.user_text_len
lock.wait_ms
session.message_count_before
session.message_count_after
```

This is useful because you can see whether a session was new, how many messages were already in memory, and whether the request had to wait for the session lock.

The lock matters because two messages for the same session should not update the same message history at the same time.

8. Engine creates an `engine.respond` span

Inside the session layer, this runs:

Python Run

That goes into:

```
backend/agent.py
```

The engine starts collecting tool records:

Python Run

This is from:

```
backend/observability/tool_context.py
```

This uses a `ContextVar`.

Think of it like:

```
For this one request, keep a private list of tool calls.
```

That list starts empty:

Python Run

Then the engine starts a span:

Python Run

This creates:

```
engine.respond
```

It records things like:

```
session.id
message.history_count
message.user_text_len
engine.time_ms
llm.calls
tool.rounds
reply.length
tool.call_count
tool.total_duration_ms
tool.success_count
tool.error_count
```

9. Actual agent call gets an `engine.agent_invoke` span

Inside `engine.respond`, the actual Deep Agent call happens here:

```
with TRACER.start_as_current_span("engine.agent_invoke") as invoke_span:
    result = await self.agent.ainvoke(...)
```

So you get a child span:

```
engine.agent_invoke
```

This is the section where the LLM decides whether to answer directly or call tools.

If the model calls no tools, you still get:

```
/chat request
  session.chat
    engine.respond
      engine.agent_invoke
```

If the model calls tools, then additional tool spans appear under this trace.

10. Every real tool call gets a `tool.invoke ...` span

When the agent calls a tool, it is actually calling the wrapped tool.

That wrapper is in:

```
backend/observability/tool_wrapper.py
```

For a sync tool, this function runs:

Python Run

For an async tool, this function runs:

Python Run

Both do the same thing.

They create a span:

Python Run

So if the agent calls the `add` tool, you get:

```
tool.invoke add
```

That span records:

```
tool.index
tool.name
mcp.alias
tool.call_id
tool.input.size
tool.success
tool.duration_ms
tool.output.size
```

If the tool errors, it records:

```
tool.success = false
error.type
error.message
```

And it also calls:

Python Run

So exceptions show up in the trace.

11. Tool calls are also stored in a request-local list

The wrapper does not only create a span.

It also appends a `ToolCallRecord`:

```
append_tool_record(
    ToolCallRecord(...)
)
```

That model is defined in:

```
backend/observability/models.py
```

A single record looks conceptually like:

```
ToolCallRecord(
    index=0,
    name="add",
    alias="core",
    success=True,
    duration_ms=12.5,
    input_size=50,
    output_size=3,
)
```

This is important because spans are for tracing systems, but the app also needs simple metrics to return in the API response and write to JSONL.

So the tool wrapper feeds two observability outputs:

```
tool wrapper
  ├── OpenTelemetry span
  └── ToolCallRecord in ContextVar
```

12. Engine summarizes the tool records

After the agent finishes, `backend/agent.py` does:

```
records = get_tool_records()
tools_used = [record.name for record in records]
tool_summary = summarize_tool_records()
```

This produces things like:

```
tool.call_count
tool.total_duration_ms
tool.max_duration_ms
tool.success_count
tool.error_count
tool.names
tool.unique_names
```

Then the engine attaches that summary to the `engine.respond` span:

```
for key, value in tool_summary.to_span_attributes().items():
    span.set_attribute(key, value)
```

So individual tool spans show detailed per-call data, while `engine.respond` has the summary.

13. Engine also adds tool events

The engine adds three types of span events:

```
_add_tool_request_events(span, messages)
_add_tool_completion_events(span, records)
```

Python Run

These are not separate spans.

They are events attached to the `engine.respond` span.

Think of them like timeline notes.

`tool.requested`

This means:

```
The LLM requested a tool call.
```

`tool.completed`

This means:

```
The actual wrapped tool finished running.
```

`tool.result`

This means:

```
A tool result message appeared in the agent transcript.
```

This helps you compare:

```
What the model asked for
vs.
What actually ran
vs.
What result came back
```

14. Session returns the result to the API layer

Back in:

```
backend/session_store.py
```

The session layer updates the saved conversation state:

Python Run

Then it returns:

```
ChatResult(
    reply=result.reply,
    tools_used=result.tools_used,
    metrics=result.metrics.model_copy(update={"lock_wait_ms": lock_wait_ms}),
    trace_id=current_trace_id(),
)
```

So now the API layer has:

```
reply
tools_used
metrics
trace_id
```

15. API layer writes one JSONL telemetry row

Back in:

```
backend/main.py
```

The `finally` block always runs, whether the request succeeds or errors.

It calculates total request time:

Python Run

It adds final attributes to the request span:

```
request_span.set_attribute("chat.success", success)
request_span.set_attribute("chat.total_response_time_ms", total_ms)
request_span.set_attribute("chat.tools_used", ",".join(tools_used))
request_span.set_attribute("chat.tool_call_count", engine_metrics.tool_calls)
```

Then it writes a local metrics row:

```
app.state.metrics.write(
    ChatTelemetryEvent(...)
)
```

That writer lives here:

```
backend/observability/metrics.py
```

It appends to:

```
metrics/chat_metrics.jsonl
```

So even if OpenTelemetry is disabled, you still get local JSONL metrics.

That is useful for quick debugging.

The trace shape

For a request that uses one tool, the trace looks like this:

```
POST /chat
└── session.chat
    └── engine.respond
        └── engine.agent_invoke
            └── tool.invoke add
```

For a request that uses two tools:

```
POST /chat
└── session.chat
    └── engine.respond
        └── engine.agent_invoke
```

```
            ├── tool.invoke add
            └── tool.invoke count_quoted_chars
```

Startup also has spans like:

```
engine.start
└── mcp.get_all_tools
    └── mcp.ensure_alias_loaded
```

The main observability files

`backend/observability/otel.py`

Sets up OpenTelemetry.

Responsible for:

```
trace provider
console exporter
OTLP exporter
FastAPI instrumentation
current trace id helper
```

`backend/observability/tool_wrapper.py`

Most important observability file.

Responsible for:

```
wrapping each MCP tool
creating tool.invoke spans
recording duration
recording success/error
appending ToolCallRecord
```

`backend/observability/tool_context.py`

Keeps request-specific tool records.

Responsible for:

```
start_tool_collection()
append_tool_record()
get_tool_records()
summarize_tool_records()
stop_tool_collection()
```

This is what lets the app say:

```
This request used 2 tools.
They were add and subtract.
Total tool time was 20 ms.
```

`backend/observability/models.py`

Defines Pydantic telemetry models.

Responsible for:

```
ToolCallRecord
ToolSummary
EngineMetrics
ChatTelemetryEvent
```

This keeps your observability data typed and consistent.

`backend/observability/metrics.py`

Writes the local JSONL file.

Responsible for:

```
metrics/chat_metrics.jsonl
```

The most important idea

The code observes tools at the boundary where tools are loaded from MCP:

Python Run

```python
wrapped_tools = [wrap_tool_for_tracing(tool, alias=alias) for tool in raw_tools]
```

That means you do **not** need observability code inside every MCP server.

Your tool servers stay simple:

Python Run

```python
@mcp.tool
def add(a: float, b: float):
    return a + b
```

The backend wraps the tool once, then every real tool execution automatically gets:

```
a child span
a duration
success/error status
input/output size
a ToolCallRecord
JSONL metrics
API response metrics
```

So the observability design is:

```
Keep tool servers clean.
Instrument tools once at load time.
Collect per-request records with ContextVar.
Summarize those records in the engine.
Write final request metrics in the API layer.
```